

# 计算物理作业 9

杨远青 22300190015

CompPhys 24

2024 年 12 月 20 日

破晓时分。

## 1 题目 1：超球体积计算

### 1.1 题目描述

The interior of a  $d$ -dimensional hypersphere of unit radius is defined by the condition  $x_1^2 + x_2^2 + \cdots + x_d^2 \leq 1$ . Write a program that finds the volume of a hypersphere using a Monte Carlo method. Test your program for  $d = 2$  and  $d = 3$  and then calculate the volume for  $d = 4$  and  $d = 5$ , compare your results with the exact results.

### 1.2 程序描述

本程序用于估计  $n$  维单位超球体的体积，并通过蒙特卡洛方法进行比较分析。主要功能包括：

- 动画演示 2 维和 3 维下的蒙特卡洛采样过程。
- 使用简单的均匀采样和重要性采样两种方法对 1 到 20 维的单位超球进行体积估计，并与理论值比较。
- 在高维数据下通过 PCA 降维进行可视化，并绘制误差、计算时间等分析图表。

在这里， $n$  维单位超球体指的是半径为 1 的  $n$  维球体（n-ball），即集合  $\{x \in \mathbb{R}^n : \|x\| \leq 1\}$  其理论体积公式为：

$$V_n = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2} + 1)}$$

#### 1.2.1 简单均匀采样

在  $\mathbb{R}^n$  空间中从  $[-1, 1]^n$  的均匀分布中采样 `num_samples` 个点，统计其中有多少点在单位超球体内，计算落入比例与超立方体的体积  $2^n$ ，从而估计体积。该方法在高位空间收敛较慢，误差较大，因为高维空间中单位超球体的体积占整个超立方体的比例很小，采样集中在浅表面。

#### 1.2.2 重要性采样

本程序的重要性采样使用多元正态分布  $N(0, \sigma^2 I)$  作为提议分布  $q(x)$  进行采样，即

$$q(x) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)$$

采样后统计各点到原点的距离，定义指示函数  $f(x) = 1_{B_n}(x)$  表示该点是否在单位球内（在则为 1，否则 0）。该提议分布采样的点集中在原点附近，更容易采样到单位球内的点，从而提高采样效率。若定义相对于 *Lebesgue* 测度  $dx$  的目标分布  $p(x) \equiv 1$ （其不满足归一化，故不是狭义的概率分布），则可将体积估计问题转化为期望值问题：

$$\int f(x)p(x)dx = \int f(x) \cdot 1 dx = \int_{\mathbb{R}^n} f(x)dx = V_n$$

重要性采样通常要求良好概率分布的提议分布  $q(x)$ ，满足  $\int q(x)dx = 1$ ，将难以直接采样的期望问题进行转化

$$\mathbb{E}_p[f(X)] = \int f(x)\frac{p(x)}{q(x)}q(x)dx = \mathbb{E}_q\left[f(X)\frac{p(X)}{q(X)}\right]$$

本题的目标分布为常函数 1，故只需要如下估计超球体积

$$\hat{V}_n = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{q(x_i)}$$

本程序还有一些辅助的可视化工具，在此不一一赘述。用户在 `src` 目录下运行 `python -u hypersphere.py` 即可运行程序，需安装辅助计算的 `numpy` 库与绘图用的 `matplotlib` 库，对于重要性采样，因追求较高精度，默认采样点数较多，故运行时间较长，故建议安装 `tqdm` 库以显示进度条。在主菜单中，用户可以选择

- (1) 动画演示（仅 2 维或 3 维）
- (2) 简单均匀采样分析（1 20 维），并可选择可视化采样点。
- (3) 重要性采样分析（1 20 维），并可选择可视化采样点。

在二级菜单中还将可以自定义一些参数，如采样点数、最大计算维度，提议分布标准差与试验次数等，详见下文的伪代码与结果示例。

### 1.3 伪代码

Powered by [L<sup>A</sup>T<sub>E</sub>X pseudocode generator](#)

---

#### Algorithm 1: Main Program

---

**Input:**  $max\_dim, num\_samples, num\_trials, method, \sigma$  (optional)

**Output:** Results for dimensions 1 to  $max\_dim$

```

1 for  $d \leftarrow 1$  to  $max\_dim$  do
2    $exact \leftarrow exact\_volume(d)$ ;  $estimates \leftarrow []$ ;
3   for  $trial \leftarrow 1$  to  $num\_trials$  do
4     if  $method = uniform$  then
5        $v \leftarrow monte\_carlo\_uniform(d, num\_samples)$ ;
6     end
7     else
8        $v \leftarrow importance\_sampling\_volume(d, num\_samples, \sigma)$ ;
9     end
10     $estimates.append(v)$ ;
11  end
12   $mean, std, rel\_err \leftarrow analyze(estimates, exact)$  ; // Compute statistics
13 end
14 return All results;
```

---

---

**Algorithm 2: Uniform Sampling**

---

**Input:**  $n, num\_samples$ **Output:** Estimated volume

- 1  $points \leftarrow \text{random uniform in } [-1, 1]^n$ ;
  - 2  $distances \leftarrow ||points||^2$ ;
  - 3  $count \leftarrow \text{sum}(distances \leq 1)$ ;
  - 4 **return**  $2^n \cdot (count/num\_samples)$ ;
- 

---

**Algorithm 3: Importance Sampling**

---

**Input:**  $n, num\_samples, \sigma$ **Output:** Estimated volume

- 1  $samples \leftarrow \text{normal}(mean = 0, cov = \sigma^2 I_n, num\_samples)$ ;
  - 2  $distances \leftarrow ||samples||^2$ ;
  - 3  $f_x \leftarrow (distances \leq 1); q_x \leftarrow (2\pi\sigma^2)^{-n/2} \cdot e^{-distances/(2\sigma^2)}$ ;
  - 4  $weights \leftarrow f_x/q_x$ ;
  - 5 **return**  $\text{mean}(weights)$ ;
- 

## 1.4 结果示例

### 1.4.1 简单均匀采样

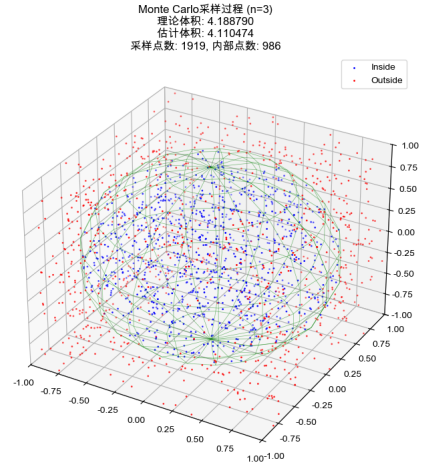
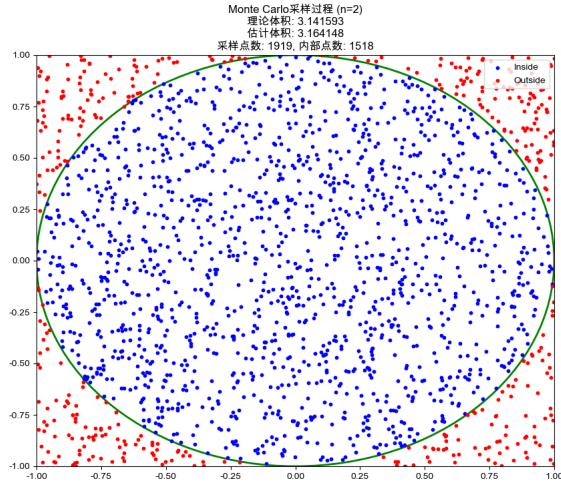


图 1: 二维与三维简单采样动画截图



n维单位超球体体积分析 (简单均匀采样)

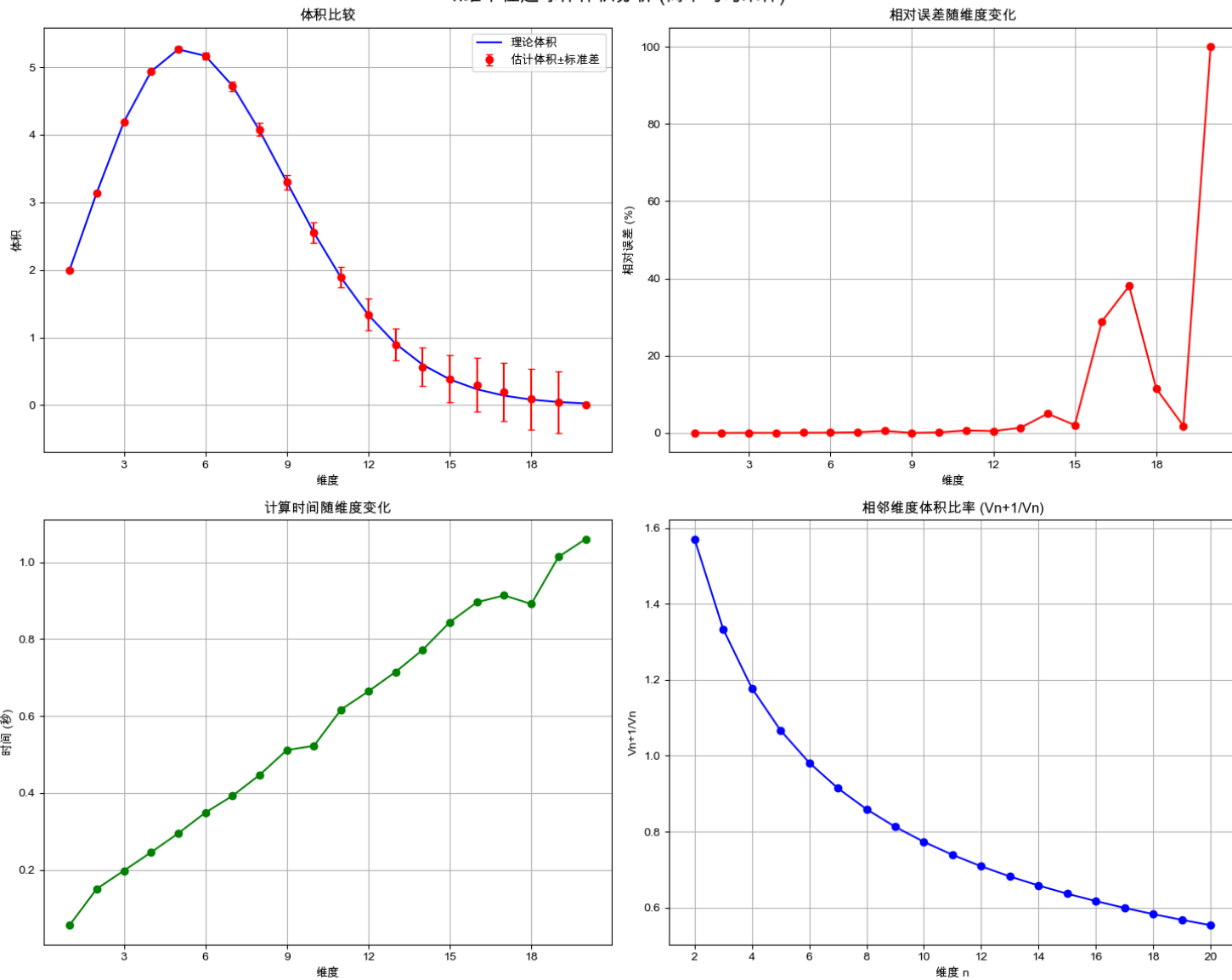


图 4: 简单均匀采样结果分析

可见在高维还是表现不容乐观的，需要借助重要性采样。

```

(base) gilbert@gilbert-YoungMacBook src % python hypersphere.py

=== 高维单位超球体 (n-ball) 体积估计程序 ===
请选择操作：
1. 动画演示采样过程 (适用于n=2,3)
2. 简单均匀采样分析 (适用于n=1~20)
3. 重要性采样分析 (适用于n=1~20)
4. 退出

请输入选择 (1/2/3/4) : 3
请输入最大维度 (1~20) [默认20]:
请输入采样的样本数量 [默认1919810]:
请输入提议分布的标准差 [默认0.5]:
请输入试验次数 [默认100]:

分析中...
100% | 20/20 [15:31<00:00, 46.57s/it]
2024-12-02 19:34:24.001 python[1853:17902] +[IMKClient subclass]: chose IMKClient_Modern
2024-12-02 19:34:24.001 python[1853:17902] +[IMKInputSession subclass]: chose IMKInputSession_Modern
2024-12-02 19:34:37.995 python[1853:17902] error messaging the mach port for IMKCFRunLoopWakeUpReliable
2024-12-02 19:34:38.396 python[1853:17902] The class 'NSSavePanel' overrides the method identifier. This method is implemented by class 'NSWindow'

详细结果：

```

| 维度 | 理论体积         | 估计体积         | 相对误差(%) | 计算时间(s) |
|----|--------------|--------------|---------|---------|
| 1  | 2.000000e+00 | 1.999995e+00 | 0.00    | 5.239   |
| 2  | 3.141593e+00 | 3.141818e+00 | 0.01    | 13.018  |
| 3  | 4.188790e+00 | 4.188722e+00 | 0.00    | 18.405  |
| 4  | 4.934802e+00 | 4.934436e+00 | 0.01    | 22.586  |
| 5  | 5.263789e+00 | 5.263612e+00 | 0.00    | 23.601  |
| 6  | 5.167713e+00 | 5.167596e+00 | 0.00    | 27.892  |
| 7  | 4.724766e+00 | 4.724776e+00 | 0.00    | 30.678  |
| 8  | 4.058712e+00 | 4.059015e+00 | 0.01    | 35.087  |
| 9  | 3.298509e+00 | 3.297827e+00 | 0.02    | 41.506  |
| 10 | 2.550164e+00 | 2.549947e+00 | 0.01    | 46.036  |
| 11 | 1.884104e+00 | 1.883534e+00 | 0.03    | 48.678  |
| 12 | 1.335263e+00 | 1.334850e+00 | 0.03    | 62.208  |
| 13 | 9.106288e-01 | 9.118455e-01 | 0.13    | 63.531  |
| 14 | 5.992645e-01 | 5.988328e-01 | 0.07    | 53.220  |
| 15 | 3.814433e-01 | 3.814017e-01 | 0.01    | 54.355  |
| 16 | 2.353306e-01 | 2.355565e-01 | 0.10    | 74.189  |
| 17 | 1.409811e-01 | 1.409728e-01 | 0.01    | 63.344  |

The figure consists of two plots. The left plot is a histogram titled '5维空间中点到原点的距离分布' (Distribution of distances from the origin in 5D space). The x-axis is labeled '距离' (Distance) and ranges from 0.2 to 1.0. The y-axis is labeled '频数' (Frequency) and ranges from 0.0 to 2.5. The histogram shows a distribution of distances, with a peak around 0.9. The right plot is a scatter plot titled '重要性采样生成的5维点的2D投影 (解释方差比: 41.02%)' (2D projection of 5D points generated by importance sampling (Explained variance ratio: 41.02%)). The x-axis is labeled '主成分1' (Principal Component 1) and ranges from -1.0 to 1.0. The y-axis is labeled '主成分2' (Principal Component 2) and ranges from -1.00 to 1.00. The scatter plot shows a dense cloud of points centered around the origin, with a slight elongation along the x-axis.

可见采样点明显向原点附近区域迁移。

n维单位超球体体积分析 (重要性采样)

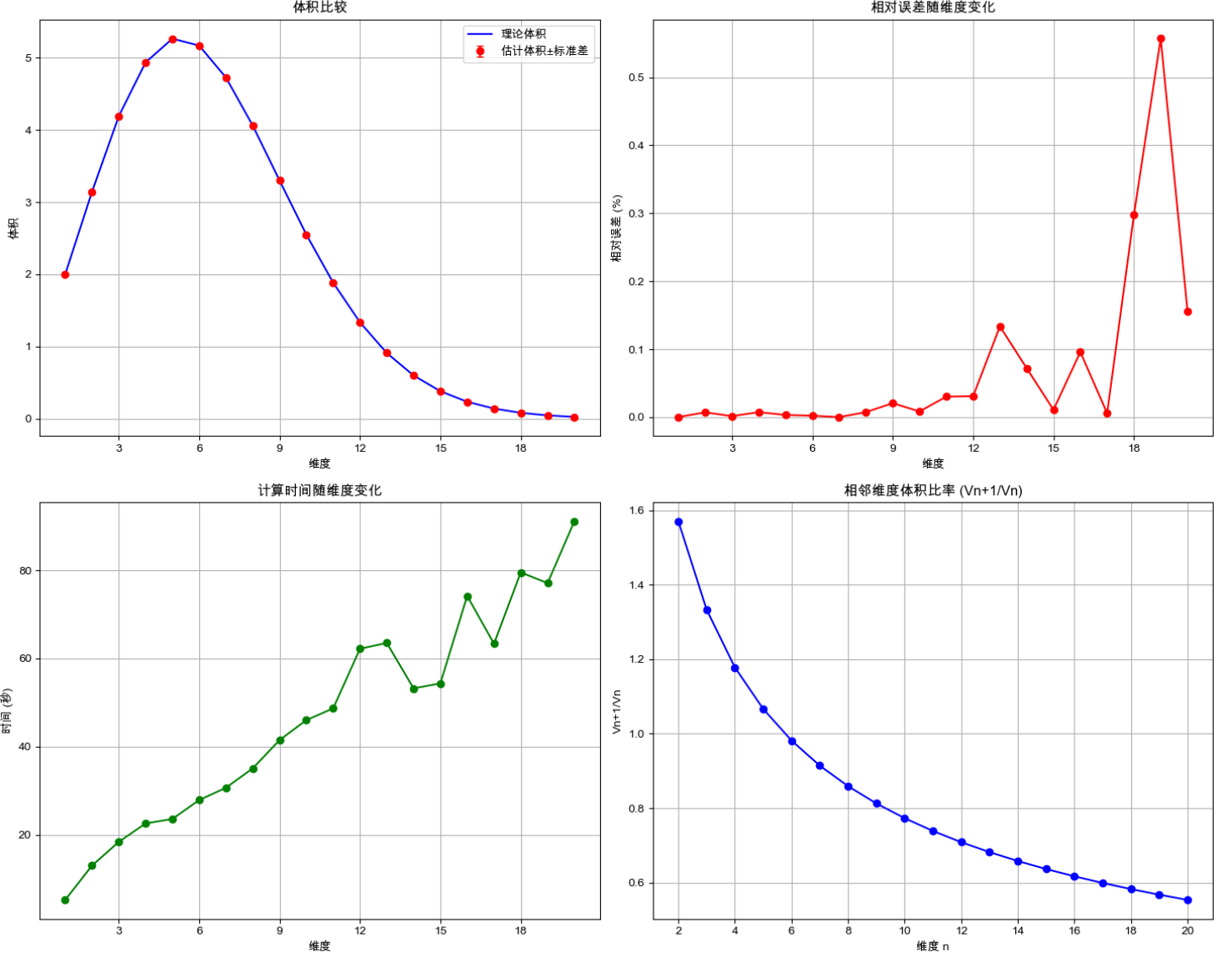


图 7: 重要性采样结果分析

相对误差以解析解为标准，最大值仅为千分之十三，相对于简单均匀采样有了质的提升。计算时间大幅增加是因为不小心将采样点数从 114514 提升至 1919810，增加了一个数量级以上。对于不同维度，还可以精细调整提议分布标准差以获得更好的效果。

## 2 题目 2: 海森堡模型

### 2.1 题目描述

Write a MC code for a 3D Face-Centered Cubic lattice using the Heisenberg spin model (adopt periodic boundary condition). Estimate the ferromagnetic Curie temperature.

$$H = -J \sum_{\langle i,j \rangle} \vec{S}_i \cdot \vec{S}_j, \quad J = 1, \quad |\vec{S}_i| = 1$$

## 2.2 程序描述

代码尚有不少改进空间，奈何期末有心杀贼，无力回天

程序内置以下模块：

- **PhysicalQuantities.py**: 管理所有的物理量与转换关系，其中比热与磁化强度均按照涨落公式计算, Binder 比用于辨别相变点与性质

$$C = \frac{\langle E^2 \rangle - \langle E \rangle^2}{k_B T^2}, \quad \chi = \frac{\langle M^2 \rangle - \langle M \rangle^2}{k_B T}, \quad U = 1 - \frac{\langle M^4 \rangle}{3(\langle M^2 \rangle)^2}$$

内有 `to_dict` 等配套方法

- **HeisenbergFCC.py**: 实现了一个 3D 面心立方晶格的海森堡模型，其中包含了初始化、构建近邻列表、测量物理量与利用 FFT 测量自旋关联函数以及结构因子等方法
- **updater.py**: 内置 `UpdaterBase` 基类，下辖 `SingleSpinUpdate`、`SwendsenWangUpdate` 与 `WolffUpdate` 三个子类，分别实现了单自旋更新、Swendsen-Wang 翻转与 Wolff 翻转三种更新方法，其中 Swendsen-Wang 与 Wolff 的团簇激活概率均为 2D Ising 模型的直接推广<sup>1</sup>

$$p = 1 - \exp[-2\beta(\vec{r} \cdot \vec{S}_i)(\vec{r} \cdot \vec{S}_j)]$$

其中  $\vec{r}$  为每步随机选择的单位投影方向，Swendsen-Wang 每次以概率  $\frac{1}{2}$  翻转所有团簇，Wolff 随机生长并翻转

- **FSAnalysis.py**: 有限尺寸分析模块，内含数据坍塌处理与临界温度定位，但在主程序中尚未集合运用，咕咕咕
- **Simulation.py**: 模拟控制类，内置单温度<sup>2</sup>模拟、温度区间模拟退火，均支持并行计算
- **Visualization.py**: 可视化模块，内含自旋构型、物理量、关联函数与结构因子的可视化方法，支持动画输出
- **Interactive.py**: 交互模块，借助 PyQt5 实现了一个简单的 GUI，集成模拟控制与可视化功能，在终端借助 `tqdm` 实时输出进度条

以上模块化代码均存放在 `src/modules` 中，可能需要预先加载到 Python 环境中，为便于助教老师调试，在 `src` 中还放置了一个 `heisenberg.py`，里面包含所有模块的代码，不便于阅读，但可在该目录直接运行 `python -u heisenberg.py` 以启动交互模拟，需要安装辅助运算的库 `numpy` 与 `scipy`，用于绘图的 `matplotlib` 与用于交互的 `pyqt5`，以及进度条 `tqdm`。

## 2.3 伪代码

Powered by [L<sup>A</sup>T<sub>E</sub>X pseudocode generator](#)

---

<sup>1</sup>Nonomura, Y. and Tomita, Y., 2015. Nonequilibrium behaviors of 3D Heisenberg model in the Swendsen-Wang algorithm. arXiv preprint [arXiv:1508.05218](#).

<sup>2</sup>本程序所有温度采用  $J/k_b$  单位



---

**Algorithm 4:** Single Spin Metropolis Update

---

**Input:** model,  $\Delta\theta_{\max}$ ,  $\beta$ **Output:** Updated model, acceptance rate

```
1  $\Delta\theta_{\max} \leftarrow \min(\Delta\theta_{\max}, \pi)$ ;
2 n_proposed, n_accepted  $\leftarrow 0, 0$ ;
3 Function RandomRotation( $s$ ):
4   Sample  $\Delta\theta \sim [0, \Delta\theta_{\max}]$ , unit vector axis;
5   Compute new spin via Rodrigues' formula;
6   return normalized spin
7 Function Update():
8    $i \sim \text{Uniform}(0, \text{model}.N)$ ,  $s_{\text{old}} \leftarrow \text{model}.\text{GetSpin}(i)$ ;
9    $E_{\text{old}} \leftarrow \text{model}.\text{CalculateEnergy}(i)$ ;
10   $s_{\text{new}} \leftarrow \text{RandomRotation}(s_{\text{old}})$ ;
11   $\text{model}.\text{spins}[i] \leftarrow s_{\text{new}}$ ,  $E_{\text{new}} \leftarrow \text{model}.\text{CalculateEnergy}(i)$ ;
12   $\Delta E \leftarrow E_{\text{new}} - E_{\text{old}}$ ;
13  n_proposed += 1;
14  if  $\Delta E \leq 0$  or  $e^{-\beta\Delta E} > \text{rand}(0, 1)$  then n_accepted += 1, model.energy +=  $\Delta E$  ;
15  else model.spins[i]  $\leftarrow s_{\text{old}}$ ;
16 Function Sweep():
17   for 1 to model.N do
18     Update();
19   end
20 Function AdjustStepSize( $r_{\text{target}}, \epsilon$ ):
21   if n_proposed = 0 then
22     return;
23   end
24   Compute  $r = \frac{\text{n\_accepted}}{\text{n\_proposed}}$ ;
25   if  $|r - r_{\text{target}}| > \epsilon$  then
26      $\Delta\theta_{\max} *= r/r_{\text{target}}$ , clamp to  $[10^{-5}, \pi]$ ;
27   end
28   Reset n_proposed and n_accepted;
```

---

---

**Algorithm 5:** Swendsen-Wang Cluster Update Algorithm (Simplified)

---

**Input:** model,  $\beta$ ,  $J$

**Output:** Updated model with new spin configuration and energy

```
1 labels  $\leftarrow$  Initialize for all spins;
2 projection_dir  $\leftarrow$  GenerateProjectionDirection() ;           // Generate random projection direction
3 Function GenerateProjectionDirection():
4   | Sample  $\mathbf{r}$  from  $\mathbb{R}^3$ , return  $\mathbf{r}/\|\mathbf{r}\|$ ;
5 Subroutine ClusterFormation:
6   | foreach neighbor pair  $(i, j)$  in neighbors do
7   |   |  $S_i, S_j \leftarrow$  model.spins[ $i$ ]  $\cdot$  projection_dir, model.spins[ $j$ ]  $\cdot$  projection_dir;
8   |   | if  $S_i S_j > 0$  and  $\text{random}() < 1 - e^{-2 \cdot \beta J \cdot S_i S_j}$  then
9   |   |   | Union( $i, j$ );
10  |   | end
11  | end
12 Subroutine ClusterUpdate:
13  | foreach cluster in labels do
14  |   | if  $\text{random}() < 0.5$  then
15  |   |   | Flip all spins in the cluster along projection_dir();
16  |   | end
17  | end
18 Function SwendsenWangUpdate:
19  | Reset labels() ;ClusterFormation() ;                               // Form clusters
20  | Extract clusters from labels() ClusterUpdate() ;                   // Flip clusters
21  | model.energy  $\leftarrow$  CalculateEnergy() ;                           // Update energy
```

---

---

**Algorithm 6:** Wolff Single-Cluster Update Algorithm

---

**Input:** model,  $\beta$ ,  $J$

**Output:** Updated model with new spin configuration and energy

1 **Function** GenerateProjectionDirection():

2     Sample random vector  $\mathbf{r}$  from  $\mathbb{R}^3$ ;

3     **return**  $\mathbf{r}/\|\mathbf{r}\|$ ;

4 **Function** WolffUpdate:

5     projection\_dir  $\leftarrow$  GenerateProjectionDirection() ;     // Generate random projection direction

6     start\_site  $\sim$  Uniform(0, model.N);

7     cluster  $\leftarrow \{start\_site\}$ , stack  $\leftarrow [start\_site]$ ;

   // Construct the cluster using DFS

8     **while** stack *not empty* **do**

9         current  $\leftarrow$  stack.pop();

10          $S_{\text{proj}} \leftarrow \text{model.spins}[current] \cdot \text{projection\_dir}$ ;

11         **foreach** neighbor in neighbors [current] **do**

12             **if** neighbor  $\notin$  cluster **then**

13                  $S'_{\text{proj}} \leftarrow \text{model.spins}[neighbor] \cdot \text{projection\_dir}$ ;

14                 **if**  $S_{\text{proj}}S'_{\text{proj}} > 0$  **and** random()  $< 1 - e^{-2\beta JS_{\text{proj}}S'_{\text{proj}}}$  **then**

15                     stack.append(neighbor);

16                     cluster.add(neighbor);

17                 **end**

18             **end**

19         **end**

20     **end**

   // Flip the cluster spins

21     **foreach**  $i \in$  cluster **do**

22          $S_{\text{proj}} \leftarrow \text{model.spins}[i] \cdot \text{projection\_dir}$ ;

23          $\text{model.spins}[i] \leftarrow \text{model.spins}[i] - 2 S_{\text{proj}} \text{projection\_dir}$ ;

24     **end**

25     model.energy  $\leftarrow$  CalculateEnergy() ;

   // Recalculate total energy

---

2.4 结果示例

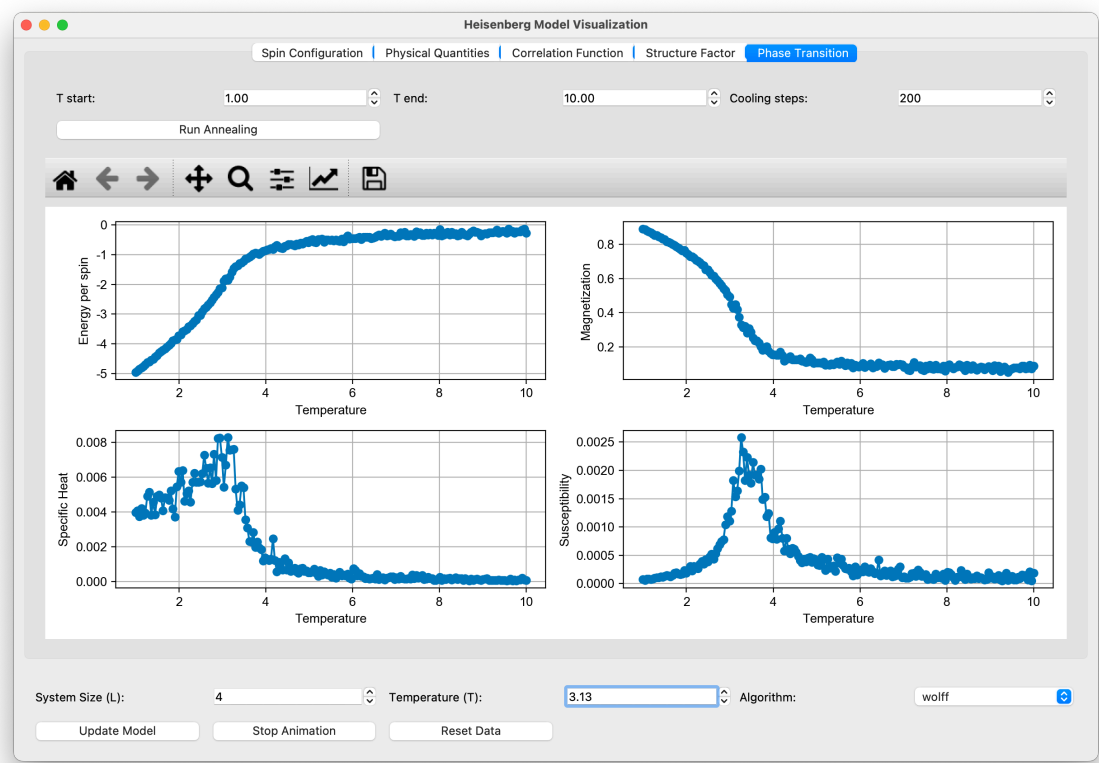


图 8: 交互窗口

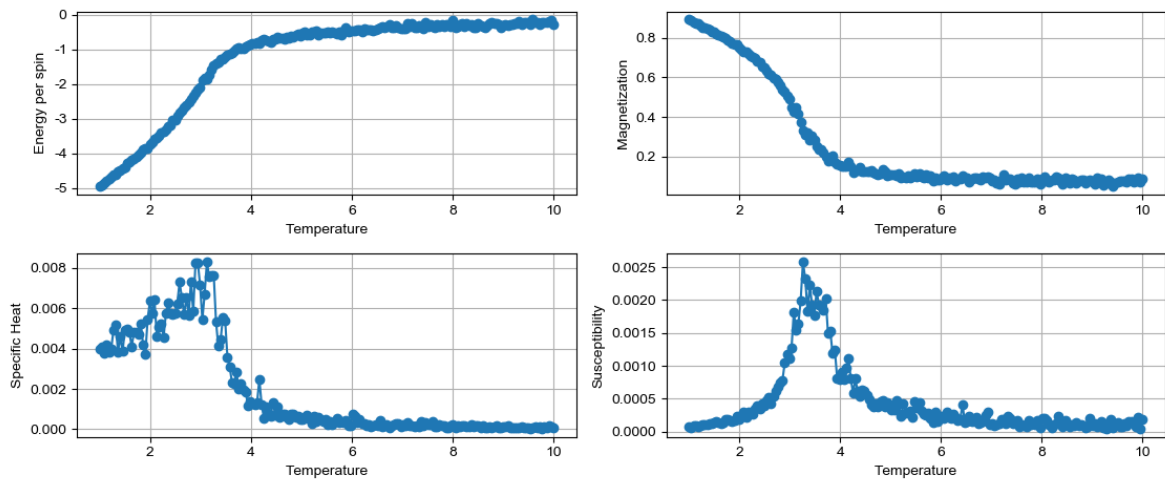


图 9: 小规模测试初步结果，采用 Wolff 更新

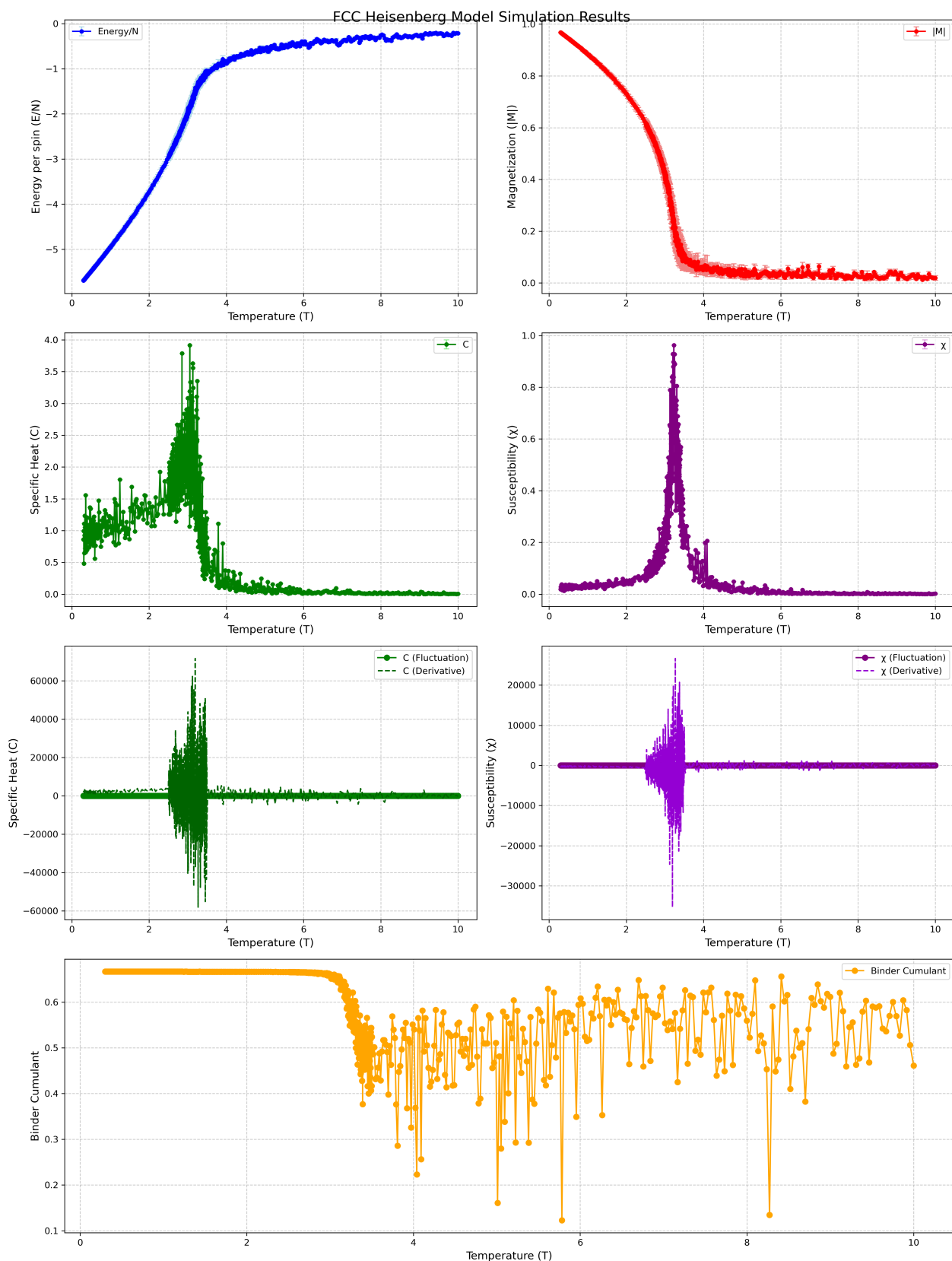


图 10: 大规模测试结果, 采用 Wolff 更新,  $8 \times 8 \times 8$  个 cell, 每个 cell 4 个自旋, 从 10 模拟退火到 0.1, 1000 个温度点, 每个点 200 热化 (临界区翻倍) + 1000mcs 采样

完结撒花, 谢谢大家!